GeoZigzag Studio: A Reproducible Workflow for Agricultural Coverage and Semantic Georouting

Luis Prieto-Lopez, Sergio Sanchez de La Fuente, Miguel A. Gonzalez-Santamarta
Angel Manuel Guerrero-Higueras, Lidia Sanchez-Gonzalez, Francisco J. Rodriguez-Lera
University of Leon, Leon, Spain
luisprietolopez@protonmail.com

***Abstract*—Agricultural mission preparation spans two different spatial tasks: dense coverage of a field and sparse travel between geographically referenced targets. Existing workflows commonly prepare these tasks in separate tools, which complicates parameter tracking, route comparison, validation, and robot- oriented export. This paper presents GeoZigzag Studio, a reproducible workflow that combines boustrophedon coverage planning with semantic georouting in one web interface and Python evaluation package. Semantic georouting is defined as routing an ordered set of georeferenced targets while semantic land-cover labels influence path cost or validity. The implementation formalizes local projection, sweep-line coverage, direct interpolation, semantic-cost A*, forbidden-zone checks, optional OSRM routing, heading generation, and ROS-style CSV/YAML export. A fixed-seed benchmark evaluates three fields and three mission scenarios. The fields span 141–3289 m^2 and generate 224–710 waypoints. Across routing scenarios, direct paths intersect 4–7 forbidden-zone segments; cost-aware paths reduce this count to zero with 4.6–7.3% additional distance. Cached OSRM routes expose maximum input-to-network snap distances of 35.2–90.8 m, demonstrating why network routes require explicit validation. The repository provides offline tests, versioned scenarios, cached service responses, generated figures and tables, and a single reproduction command.**

***Index Terms*—agricultural robotics, coverage path planning, geospatial routing, reproducibility, waypoint preparation, ROS 2**

I. Introduction

Agricultural mobile robots operate at more than one spatial scale. Within a plot, sensing or treatment requires dense parallel passes. Between plots or resources, the mission is instead a sparse sequence of targets such as water points, crop areas, or access tracks. Coverage path planning (CPP) and geographic routing address these needs, but their inputs, controls, and output formats differ. Transferring a mission between tools can silently change coordinate order, sampling density, target order, or assumptions about traversability.

This separation also weakens experimental traceability. A screenshot alone does not identify the algorithm, route parameters, map state, or external service response that produced it. A research workflow should regenerate its routes, metrics, tables, and figures from versioned inputs. It should also distinguish a route that is geometrically valid from one that has been executed successfully by a robot.

GeoZigzag Studio addresses this preparation layer. It combines interactive field editing and mission ordering with a deterministic Python core. We use *semantic georouting* to mean: given an ordered sequence of sparse WGS84 targets and a spatial semantic model, compute a connecting route for which labels such as track, shrub, or water modify traversal cost or admissibility. This differs from CPP, whose objective is dense traversal of a bounded area, and from local robot navigation, which closes the loop using localization and perception.

The contributions are:

- a unified, explicit workflow for dense field coverage and sparse semantic mission routing;
- formal algorithms for projection, coverage, cost-aware routing, forbidden-zone validation, and ROS-oriented orientation export;
- a multi-scenario benchmark comparing direct, semantic-cost, and cached OSRM routes using distance, waypoints, turns, time, intersections, snap distance, and success rate; and
- a reproducibility package with 20 offline tests, scenario files, service fixtures, generated artifacts, and `make reproduce`.

II. Related Work and Research Gap

CPP surveys describe decomposition and back-and-forth patterns for complete area traversal [1], [2]. Agricultural CPP must additionally consider field shape, working width, headlands, and vehicle turning constraints [3], [4]. The Fields2Cover library provides reusable swath, route, and path components and a benchmarking basis [5]. Nilsson and Zhou proposed 54 standard fields and agronomic measures for systematic comparison [6]. GeoZigzag does not replace these richer vehicle-aware planners; it provides a lighter mission-preparation and reproduction layer.

General GIS tools such as QGIS can construct saved chains of vector and raster operations through the Processing Modeler [7]. OSMnx automates acquisition, projection, analysis, and routing over OpenStreetMap networks [8], while OSRM exposes network route and nearest services and reports the distance from an input coordinate to its snapped network location [9]. These tools are powerful, but they do not by themselves provide the combined agricultural coverage, semantic mission, and ROS-style export workflow considered here.

At execution time, Navigation2 provides ROS 2 planning and control and a waypoint-follower action [10], [11]. It expects a configured robot, frames, localization, costmaps,

TABLE I

WORKFLOW-LEVEL CAPABILITY COMPARISON.

Tool	CPP	Sem.	Web	Zones	ROS	Repro.
Fields2Cover	Y	–	–	P	P	Y
QGIS	P	P	–	Y	–	Y
OSMnx/OSRM	–	Y	P	P	–	Y
Nav2 waypoint follower	–	–	–	Y	Y	P
GeoZigzag Studio	Y	Y	Y	Y	Y	Y

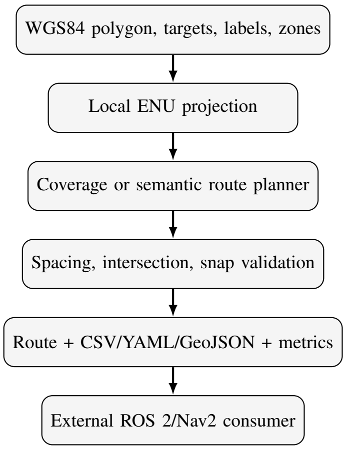

Fig. 1. GeoZigzag preparation pipeline. The external consumer is not part of the route-generation evaluation.

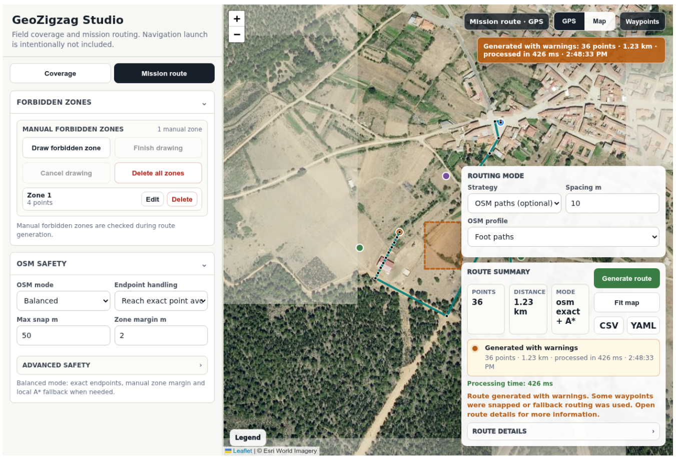

Fig. 2. Mission interface with ordered targets, routing controls, a manually defined forbidden zone, route status, and export controls.

Algorithm 1: Polygon zigzag coverage

1: validate polygon, $s_r > 0$, and $s_w > 0$
2: project vertices to local ENU and choose sweep axes
3: **for** $d = \min d_\perp$ to $\max d_\perp$ in steps s_r
4: intersect line d with all polygon edges
5: sort intersections and pair inside intervals
6: sample each interval at maximum spacing s_w
7: append samples forward/reverse on alternating sweeps
8: attach yaw/quaternion; return route and coverage metrics

and controllers; it is a downstream consumer rather than a geographic authoring environment. Table I locates the gap. "P" means possible through configuration or plugins but not a first-class workflow; "ROS" denotes waypoint export or consumption, not validated execution.

III. System and Reproducible Workflow

Figure 1 shows the software boundary. Inputs are (i) a WGS84 field polygon or labelled rectangle, (ii) row direction and start corner, (iii) positive row and waypoint spacings, (iv) an ordered set of WGS84 targets with semantic labels, (v) optional forbidden polygons, and (vi) a routing strategy. Outputs are the sampled route, yaw and planar quaternion at each point, CSV/YAML/GeoJSON files, validation status, metrics, and provenance.

The Leaflet interface makes generation explicit. Editing a field, spacing, target order, strategy, or forbidden zone marks the route *pending*, disables stale downloads, and does not silently regenerate. Generation ends in success, warning, or error state. OSRM warnings show the maximum snap distance; intersection failures and fallback details remain visible to the operator. Figure 2 shows the mission preparation view.

IV. Methods

A. Projection, Heading, and Orientation

For field-scale geometry, all input coordinates remain WGS84 while computation uses a local equirectangular ENU approximation. For latitude–longitude (ϕ, λ) and origin (ϕ_0, λ_0) in radians,

$$x = R\cos\phi_0(\lambda - \lambda_0), \quad y = R(\phi - \phi_0), \tag{1}$$

with $R = 6{,}378{,}137$ m, x east, and y north. The origin is the first polygon point for coverage and the mean input coordinate for routing. This approximation is used only for the sub-kilometre scenarios evaluated here.

For consecutive local points p_i, p_{i+1}, ROS ENU yaw is $\psi_i = \operatorname{atan2}(\Delta y, \Delta x)$ normalized to $[-\pi, \pi]$. The final point reuses the previous yaw. Exported planar orientation is $(q_x, q_y, q_z, q_w) = (0, 0, \sin(\psi/2), \cos(\psi/2))$.

B. Coverage Planner

The sweep direction is selected explicitly or inferred from the longest polygon edge. Perpendicular sweep lines are separated by row spacing s_r. Each line is intersected with every polygon edge; sorted intersections are paired into valid in-field intervals. Samples along an interval are no farther apart than waypoint spacing s_w. Traversal direction alternates by sweep.

The implementation reports polygon area, row count, mean clipped row length, waypoint count, route distance, and discrete turns. A "turn" is a heading change of at least 30°; it is a polyline metric, not a claim of kinematic feasibility.

C. Semantic and Network Routing

Direct routing samples straight segments between ordered targets. Cost-aware routing rasterizes all inputs on an 8-connected local grid. The evaluated resolution is 4–5 m. A semantic point influences a radius of two cells (four for water),

Algorithm 2: Semantic/cost-aware mission route

validate ordered targets, resolution, labels, and zones
project inputs; build padded local cost grid
assign semantic costs; mark forbidden cells ∞
for each consecutive target pair (g_i, g_{i+1})
    run 8-connected A* using cost J
    if no path: return explicit failure
    restore exact target endpoints and concatenate
test every route segment against every zone edge
attach orientation; export route, validation, and metrics

using costs $1, 1.5, 10, 30, 80, 1000$ for road, track, grass, shrub, scrub, and water. Forbidden polygon cells receive infinite cost. For a path $P = (v_0, \ldots, v_n)$, A* minimizes

$$J(P) = \sum_{k=1}^{n} c(v_k)\, \ell(v_{k-1}, v_k), \tag{2}$$

where ℓ is one or $\sqrt{2}$ grid steps. The admissible heuristic is $c_{\min}$ times Euclidean cell distance. Failure to connect a target pair is reported; there is no silent direct-route fallback.

For optional network routing, the OSRM route service returns a network geometry and snapped waypoint records. GeoZigzag records $d_{\text{snap}} = \max_i d(g_i, \hat{g}_i)$ and rejects results above a configured threshold. The offline benchmark uses a versioned response cache, not a live service, so results do not change with network availability or OSM updates.

Forbidden-zone validation counts each route segment that lies inside, touches, or crosses at least one polygon. It combines endpoint-in-polygon tests with orientation-based segment intersection, including collinear contact. A route is valid with respect to declared zones only when this count is zero.

V. Experimental Evaluation

A. Protocol

The evaluation configuration fixes seed 20260703 and executes each primary planner five times. Timing is the median wall-clock duration on an Intel Core i7-13700KF with Python 3.10; it excludes file export and plotting. Three coverage scenarios represent a compact rectangle, a medium quadrilateral, and an irregular five-vertex field. Their areas are 141.2, 932.0, and 3288.9 m^2. Three semantic scenarios use different target orders and manual forbidden-zone layouts. Every routing scenario is evaluated with direct, semantic-cost A*, and cached OSRM routing. The OSRM driving responses were captured on 3 July 2026 and retain their reported snap distances.

Metrics are waypoint count, distance, coverage rows, heading-change turns, median computation time, forbidden-zone intersections, maximum OSRM snap distance, and success rate. Coverage sensitivity crosses four row spacings $(0.5, 0.75, 1.0, 1.5)$ m with three waypoint spacings $(0.5, 1.0, 2.0)$ m.

B. Results

All 60 primary executions (12 planner–scenario combinations, five repeats) completed, for 100% software-level generation success. Coverage produced 224, 347, and 710 waypoints

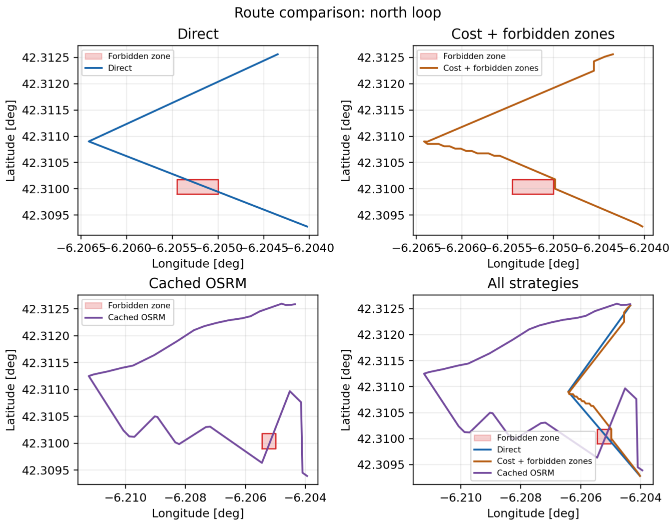

Fig. 3. Direct, forbidden-zone-aware semantic-cost, cached OSRM, and overlay views for the north-loop mission.

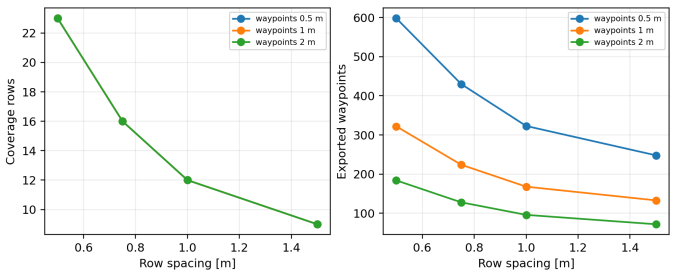

Fig. 4. Sensitivity to row and waypoint spacing on the compact field.

over 16, 27, and 36 rows, with route distances of 205.8, 682.0, and 1753.7 m. Median coverage computation remained below 1 ms on the test machine.

Table II shows the routing comparison. Direct routing is the shortest exact-target baseline but intersects declared zones. Cost-aware routing eliminates every intersection, increasing distance by 4.7%, 4.6%, and 7.3% for north-loop, west–east, and hazard-crossing missions. Its larger turn count exposes grid discretization and shows why these routes still require smoothing or a local planner before execution.

Cached OSRM follows the mapped driving network rather than the straight mission geometry. It is substantially longer in two scenarios and still intersects zones in two. More importantly, maximum snap distance is 87.7, 90.8, and 35.2 m. Thus a successful HTTP response is not sufficient evidence that a network route reaches an agricultural target. Figure 3 visualizes this difference.

Sensitivity results in Fig. 4 separate geometric coverage from sampling density. Increasing row spacing from 0.5 to 1.5 m reduces rows from 23 to 9 and distance from 291.1 to 120.6 m. At fixed 0.75 m row spacing, changing waypoint spacing from 0.5 to 2.0 m changes exported points from 430 to 128 while preserving route distance (205.8 m to numerical precision).

TABLE II
ROUTING RESULTS GENERATED BY `MAKE REPRODUCE`. "HITS" COUNTS ROUTE SEGMENTS INTERSECTING DECLARED FORBIDDEN ZONES.

Scenario	Strategy	Points	Dist. (m)	Turns	Hits	Time (ms)
north_loop	direct	54	518.2	1	4	0.05
north_loop	cost_aware	83	542.7	18	0	58.46
north_loop	osrm_cached	34	1552.0	11	1	0.02
west_east	direct	54	522.8	1	5	0.05
west_east	cost_aware	98	546.8	12	0	22.85
west_east	osrm_cached	30	1125.2	6	0	0.02
hazard_crossing	direct	42	314.7	1	7	0.04
hazard_crossing	cost_aware	66	337.7	16	0	82.37
hazard_crossing	osrm_cached	11	293.6	4	3	0.01

VI. REPRODUCIBILITY AND SOFTWARE VALIDATION

The command `make reproduce` runs 20 offline tests, the configured evaluation, figure/table generation, and LaTeX compilation. Each successful route is exported as CSV, YAML, and GeoJSON. The summary records the random seed, configuration SHA-256 digest, fixture version, per-run success, and all reported metrics. Tests cover rectangle and polygon coverage, invalid spacing, yaw/quaternion conversion, direct and cost-aware routing, impossible routes, forbidden intersections, export schemas, mocked OSRM success/failure, generated files, and web-state contracts. Online map tiles and live OSRM are optional and excluded from offline tests.

The Python modules separate geometry, coverage, routing, metrics, export, and evaluation. The previous public API remains as a compatibility facade. This keeps the web/CLI workflow stable while allowing each scientific component to be exercised independently.

VII. LIMITATIONS AND THREATS TO VALIDITY

The semantic cost layer is a controlled proxy built from labelled point neighborhoods, not a calibrated traversability map. The grid paths contain discrete corners and do not enforce turning radius, headland width, vehicle footprint, soil response, or dynamics. Manual forbidden zones are only as complete as operator input. The local projection is appropriate for the short routes studied here, not regional missions.

Cached OSRM data improve repeatability but cannot establish current network quality. Live results may change, fail, or snap differently as OSM and server profiles evolve. Timing values are machine-dependent and the scenario set is too small for claims of general superiority. The benchmark measures route preparation, not localization, control, obstacle avoidance, simulated completion, or field performance. ROS 2 waypoint execution and camera-based crop-row following require separate frame, topic, controller, and safety validation. Future work should compare against Fields2Cover on standard field sets and measure executed cross-track error, missed area, interventions, completion time, and target arrival error in simulation and on a robot.

VIII. CONCLUSION

GeoZigzag Studio turns two commonly separated planning tasks into a measurable and reproducible agricultural mission-preparation workflow. Dense boustrophedon coverage and sparse semantic georouting share projection, validation, orientation, export, and provenance. The evaluation demonstrates three concrete findings: row and waypoint spacing affect different properties; declared forbidden zones can be avoided at a measurable distance and complexity cost; and OSRM snap distance must be reported rather than hidden. The resulting artifact generates its routes, metrics, figures, and manuscript tables from versioned inputs while keeping robot execution claims outside the evidence provided.

REFERENCES

[1] H. Choset, "Coverage for robotics: A survey of recent results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, pp. 113–126, 2001.

[2] E. Galceran and M. Carreras, "A survey on coverage path planning for robotics," *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, 2013.

[3] T. Oksanen and A. Visala, "Coverage path planning algorithms for agricultural field machines," *Journal of Field Robotics*, vol. 26, no. 8, pp. 651–668, 2009.

[4] M. Höffmann, S. Patel, and C. Büskens, "Optimal guidance track generation for precision agriculture: A review of coverage path planning techniques," *Journal of Field Robotics*, vol. 41, no. 3, pp. 823–844, 2024.

[5] G. Mier, J. Valente, and S. de Bruin, "Fields2cover: An open-source coverage path planning library for unmanned agricultural vehicles," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 2166–2172, 2023.

[6] R. S. Nilsson and K. Zhou, "Method and bench-marking framework for coverage path planning in arable farming," *Biosystems Engineering*, vol. 198, pp. 248–265, 2020.

[7] QGIS Development Team, *QGIS User Guide: The Graphical Modeler*, 2026, accessed: 2026-07-03. [Online]. Available: https://docs.qgis.org/latest/en/docs/user_manual/processing/modeler.html

[8] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.

[9] Project OSRM Contributors, *OSRM HTTP API Documentation*, 2026, accessed: 2026-07-03. [Online]. Available: https://project-osrm.org/docs/v5.22.0/api/

[10] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, 2020, pp. 2718–2725.

[11] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.